The Latent Bridge: A Continuous Slow–Fast Channel for Real-Time Game Agents

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

A real-time agent for general computer use—with games as the most demanding case—must act within tens of milliseconds while still planning over seconds. These two regimes sit at opposite ends of the latency–quality tradeoff. A *reasoning* VLM (Qwen3-VL-8B-Thinking) deliberates effectively but requires ∼1.5 s per response—far too slow for a 15 Hz control loop. In contrast, a *reactive* VLM (MiniCPM-o 4.5) acts in milliseconds but underperforms on planning-heavy tasks. We couple two frozen models of matched scale (9 B reactive, 8 B reasoning), leaving the communication *channel* as the sole trainable component. The standard coupling is a **Text Bridge** (T): the slow model writes a suffix the fast model reads. We introduce a learned continuous **Latent Bridge** (L) that projects the slow model's residuals into the fast model's input-embedding space in a LLaVA-style manner, avoiding any text round-trip; both are compared against **Fast-Only** (F). On 7 Atari games and a driving domain (MetaDrive), tuning the action decoder per channel on held-out seeds, the Latent Bridge matches or beats the Text Bridge in every domain: it significantly improves two games (MsPacman +57%, RoadRunner +28%) and is a safe drop-in elsewhere. Combining both channels interferes destructively (RoadRunner −96%), so only one should be used. The benefit is highly predictable: the bridge helps if and only if slow reasoning already beats fast reaction ($T > F$)—the Latent and Text gains over Fast-Only co-vary at $r = 0.93$. MetaDrive is the controlled negative, where the Latent Bridge is provably inert because the Text Bridge adds no value. We release replay recordings and reproducible pipelines.

Code: https://github.com/19PINE-AI/latent-bridge-games
Website: https://01.me/research/latent-bridge-games

1 Introduction

We seek agents that operate a computer the way a person does—*general computer use* (GCU): read the screen, issue low-level inputs, close the loop, repeat. The most demanding form of GCU is playing real-time video games on a phone or desktop: the agent sees raw pixels and must react every few tens of milliseconds, while also pursuing goals that require planning over seconds. This integrates two opposing regimes—fast reaction and slow deliberation—that a single model struggles to reconcile.

A single model is forced to choose. Open-source multimodal LLMs come in two categories, each sacrificing one side of the latency–quality trade-off. A *reasoning* VLM (e.g. Qwen3-VL-8B-Thinking [17]) produces strong reasoning via chain of thought but requires ∼1.5 s per

response—tens of frames late for a ∼15 Hz control loop, so it simply cannot sit in the reactive path, regardless of its quality. Conversely, a *reactive* VLM (e.g. MiniCPM-o 4.5 [16]) responds in tens of milliseconds but lacks meaningful planning: this reactive model acting alone—which we call **Fast-Only** (F)—leaves much to be desired on planning-heavy games (e.g. on MsPacman, coupling in a reasoner roughly doubles the score). The latency gap is fundamental: models good enough to plan are too slow to act in real-time. The natural solution is to run *both* models—a fast reactive model in the control loop and a slow reasoning model operating asynchronously in the background, passing the slow model's deliberation to the fast model. This fast/slow split has become the emerging industry pattern for real-time interaction. For example, Thinking Machines Lab's recent *Interaction Models* [21] pair a live interaction model with an asynchronous background reasoner, coupling them by streaming a "rich context package"—i.e. shared text/context. Such systems are typically closed and jointly trained. In contrast, we couple two *frozen, open-source* general-purpose models and investigate *how* the slow model's deliberation should reach the fast model—specifically, whether a learned continuous latent channel beats the standard text-based coupling.

Text Bridge vs. Latent Bridge. The standard approach is the **Text Bridge** (T): the slow model writes its conclusion and the fast model reads it as a prompt suffix. As an alternative, we introduce the learned continuous **Latent Bridge** (L): it projects the slow model's residual stream directly into the fast model's input-embedding space (in the style of LLaVA) and prepends a small number of latent tokens—eliminating any text round-trip. We compare both against **Fast-Only** (F). Both base models are frozen, general-purpose multimodal LLMs at matched scale (9 B fast, 8 B slow). This design ensures that the *channel* itself—rather than differences in model capability—is the only variable under study; the bridge is the sole learned component. The fast model produces actions at ∼15 Hz, while the slow model deliberates at ∼1 Hz. We evaluate this coupling in domains where its effects are directly measurable: Atari [4] games, where 100 ms ghost-dodging meets 10 s route-planning, and a driving simulator (MetaDrive) as a controlled negative case.

What this paper shows.

- **Architecture matters.** A first attempt (v1, 256-d cross-attention at 2 of 36 layers) converged to KL=0.004 offline yet *failed* at deployment; the working v2 is the LLaVA pattern—project slow residuals into the fast LLM's 4096-d input-embedding space and prepend, so all layers attend via standard causal attention (§3).
- **Tuned per channel, the Latent Bridge is never significantly worse than the Text Bridge and is significantly better on 2 of 7 games** (MsPacman +57%, RoadRunner +28%; §4.4). A fixed greedy decoder (always taking the highest-probability action) flatters the Latent Bridge to 4 wins, but that edge is *greedy-specific*—it vanishes once actions are instead sampled (§4.3). Since the decoder is a tunable deployment hyperparameter, the fair test gives each channel its own best decoder, selected on held-out seeds; the two in fact prefer *different* decoders.
- **Using both channels at once *hurts*—couple via exactly one** (§4.5). Feeding text suffix and latent tokens in one pass never beats the better single channel and significantly interferes on 3 games (RoadRunner −96%): the frozen head, trained on one conditioning signal at a time, gets a worse policy from two. The Latent Bridge is the safe single-channel default.

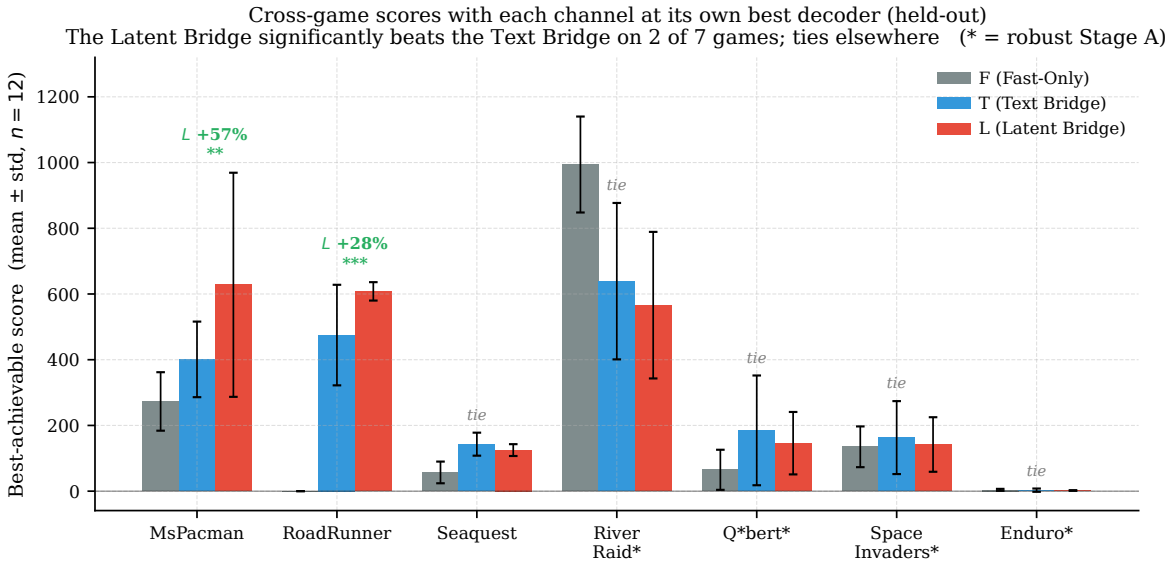

Figure 1. Cross-game scores with each channel at its own best decoder, selected on held-out seeds ($n = 12$; reported Stage-A variant per game, selected by the rule in §2). This is the fair comparison, since the action decoder is a deployment hyperparameter one tunes (§4.4). The Latent Bridge significantly beats the Text Bridge on MsPacman (+57%) and RoadRunner (+28%) and ties on the other five, never losing. Stars: L-vs-T significance by Mann–Whitney U (** $p < .01$, *** $p < .001$); $*$ on a game label marks the robust-Stage-A variant (§5). Both variants are tabulated in Appendix A.

- **When the bridge helps is a property of the task, not the channel.** Across 7 Atari games and a driving domain (MetaDrive), the Latent Bridge's benefit over Fast-Only ($L-F$) tracks the Text Bridge's ($T-F$) at Pearson $r = 0.93$: the bridge pays off exactly when slow reasoning beats reaction ($T > F$), and is inert or harmful otherwise. MetaDrive is the controlled negative: swapping the trained latent for zeros or random vectors (a *bridge-replacement control*) leaves the score unchanged, confirming the Latent Bridge is inert there (§7). We frame it this way rather than as a "text is bandwidth-limited" story, which our own ablations do not support.
- **Most collapsed cells are not bridge failures** but out-of-distribution (OOD) brittleness of the fast model's action head under the suffix/bridge inputs. Retraining that head to tolerate them fixes the collapses, dramatically so on River Raid (§5).
- **The Latent Bridge does not always beat Fast-Only.** On several games where it beats the Text Bridge, Fast-Only still beats both: the bridge improves over text *at matched architecture*; it does not always justify running a slow model at all.

The predictor above is diagnostic, not a priori: it requires measuring $T - F$ at deployment. Emission lexical diversity, our initial guess at an a priori signal, does not predict the sign of the Latent-vs-Text gap ($r = -0.08$, $n = 7$; §7.4)—kept as a negative result.

2 Setup

Models. We use two frozen multimodal models of similar scale. The *fast* (reactive) model is MiniCPM-o 4.5 [16] (9 B parameters, bf16). The *slow* (reasoning) model is Qwen3-VL-8B-

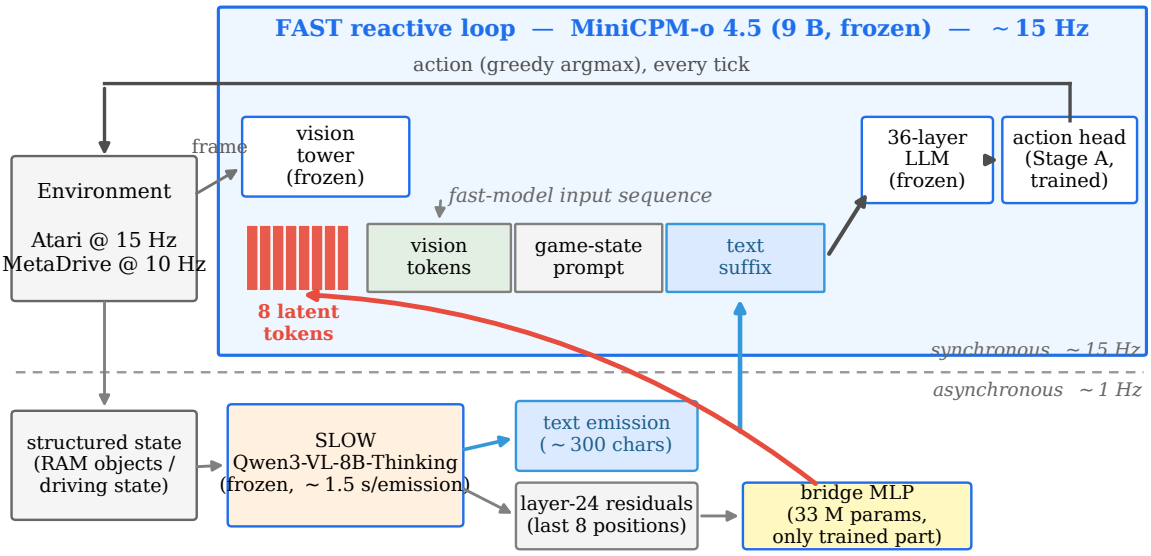

Figure 2. System architecture. The fast model (MiniCPM-o 4.5, frozen) runs the reactive loop: vision tokens and a game-state prompt feed a 36-layer LLM whose Stage-A-tuned action head emits one action per tick (∼33–38 ms warm path). The slow model (Qwen3-VL-8B-Thinking, frozen) reasons asynchronously over structured state at ∼1 Hz; the fast loop never blocks on it and reuses the latest emission until replaced. The slow output reaches the fast model two ways: its text emission appended as a prompt suffix, and its layer-24 residuals projected through the 33 M-parameter bridge MLP—the only trained component—into 8 latent tokens prepended to the input. The three strategies (Fast-Only, Text Bridge, Latent Bridge) toggle which of these the fast model receives; the strategies and the three-stage training pipeline are detailed in §2.

Thinking [17] (8 B parameters, bf16).

Tick budgets. Atari environments run at 60 Hz but the fast model is limited to a 15 Hz control rate—one action every ∼67 ms (a *tick*). The slow model produces one deliberation (an *emission*) asynchronously at roughly 1 Hz, so each emission is reused for ∼15 ticks; because its residuals are cached, the projected latent tokens are identical across those ticks until the next emission lands. Measured warm-path inference latencies (with vision caching) are $F = 33$ ms for the fast model alone and $L \approx 38$ ms when using the Latent Bridge (8 prepended tokens add ∼5 ms). End-to-end wall-clock time for the full Latent-Bridge system is dominated by GPU contention from the asynchronous slow model rather than by the bridge itself. (Our headline scores in §4 use per-tick vision rather than this cached path, so they are correctness numbers, not latency-optimized; see Appendix E.)

Three strategies compared (Figure 2). All three share the frozen fast model and its Stage A action head; they differ only in what the slow model contributes to the fast context.

- **Fast-Only** (F): the fast model acts alone, ignoring the slow model.

- **Text Bridge** (T): Fast-Only plus the slow model's full text emission appended verbatim as a prompt suffix (median 302 chars; samples in Appendix C).
- **Latent Bridge** (L): Fast-Only plus the slow model's projected latent tokens prepended (8 tokens, 4096-d each).

We write the three in prose by name and abbreviate them F, T, L in figures and formulas. (A slow-*only* policy is not a fourth strategy: it must finish a full perception–reasoning–emission cycle synchronously before each action, so at the slow model's $\sim$1.5 s per emission it acts at well under 1 Hz, missing tens of frames per decision—the speed problem that motivates the coupling.)

Training pipeline. The training has three stages:

- **Stage A**: behavioral cloning (BC) from Stable-Baselines3 (SB3) experts [18] onto the fast model's *action head*—the small head mapping the LLM's final hidden state to a game action.
- **Stage B**: Collect T-trajectories and cache (frame, slow text, slow residuals at layer 24, last 8 positions).
- **Stage C**: $\mathrm{KL}(\pi_L \,\|\, \pi_T)$ on the slow's projection only, keeping both base models frozen ($\sim$5K samples/game, final KL $\sim$0.005). Hyperparameters and per-game val-accuracies are in Appendix D.

Stage A robustness is a tuned hyperparameter (*not* per-game cherry-picking). For each game, we train two variants of the Stage A action head: *bare* (trained on plain state prompts) or *robust* (trained with suffix-probability 0.5: half of the batches get a slow-style text suffix appended to teach the head to tolerate the OOD suffix/bridge inputs it meets at deployment; §5). This is a single binary hyperparameter. We select the variant per game by the following rule: choose the variant with the higher L performance, provided its Text Bridge has not collapsed ($T > 0$, ensuring the L-vs-T comparison is non-degenerate). Ties and double-collapses are broken in favor of higher T performance. This rule is *conservative*. On Enduro, the bare variant has marginally higher L (7.8 vs 5.8) but $T = 0$, so we select the robust variant ($T = 5$, $L = 6$). This choice lowers, rather than raises, the reported L for that game. Because the rule maximizes L— the deployed quantity—and is blind to the $L - T$ *gap*, it cannot inflate the Text-vs-Latent comparison. Both variants for all games are reported in Appendix A. The robustness choice itself is informative: it *rescues* games whose bare action head collapses under suffix/bridge inputs (River Raid L: $360 \to 612$, Q*bert $0 \to 50$), but *harms* games where the bare head already works well (MsPacman $628 \to 60$, Seaquest $80 \to 0$). Importantly, the cross-game predictor (§7) remains strong even without any selection ($r = 0.93$ on the 8 reported cells, $r = 0.96$ on all 16 evaluated cells).

Evaluation. We evaluate each (game, strategy, variant) combination using 3 random seeds $\times$ 4 episodes per seed, for a total of $n = 12$ episodes per cell. Episodes are capped at 500 ticks. Arcade Learning Environment (ALE) seeds vary per episode, but action selection is greedy (the action with the highest probability). As a result, many cells exhibit zero per-episode variance (Appendix B).

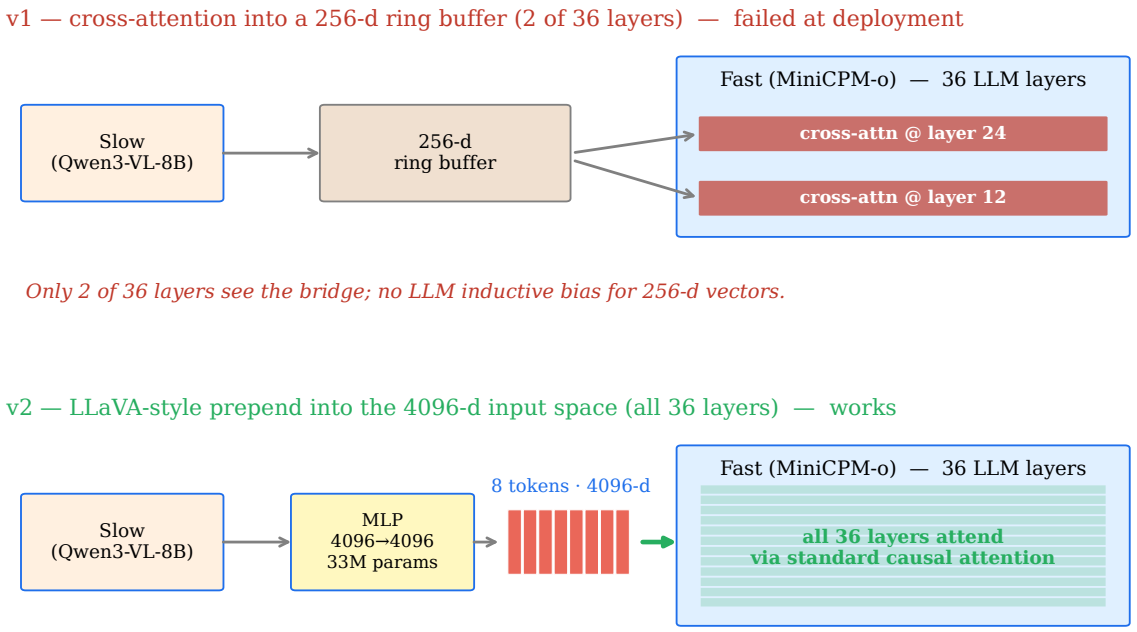

Figure 3. v1 vs v2 bridge architectures. v1 (top, failed): 256-d cross-attention at 2 of 36 layers. v2 (bottom, working): 4096-d prepend, all 36 layers attend.

Games. A total of nine Atari games were attempted. *Frostbite* is excluded since Stage A converged to random val-accuracy, so no F/T/L comparison is meaningful. *Pong* is reported for completeness but provides little insight. Stage A validation accuracy reaches only 25.1% (versus 16.7% random, or 1.5× baseline). Because the bare Fast-Only (F) policy cannot reliably move the paddle, no bridge is able to rescue its performance. The remaining seven games are MsPacman, Seaquest, SpaceInvaders, RoadRunner, River Raid, Enduro, Q*bert.

3 Architecture: v1 cross-attention failed; v2 LLaVA-style works

We tried two bridge architectures (Figure 3). v1 projected slow-model residuals into a 256-d ring buffer and injected via cross-attention at LLM layers 12 and 24 (2 of 36) of the fast model. *Despite converging to KL=0.004 on offline data*, v1 failed at deployment: the Latent Bridge scored 225 on MsPacman against Fast-Only's 256, bimodal with 4/12 catastrophic episodes. Three v1 variants (ungated, gated, gated+head-tune) all failed.

v2. We adopt the same pattern LLaVA [14], BLIP-2 [12], Flamingo [2], and MiniCPM-o [16] use to couple vision and language: project slow residuals into the fast LLM's input embedding space (4096-d) via a 33 M-parameter 2-layer MLP, and prepend the resulting 8 tokens to the fast model's input sequence. All 36 LLM layers attend over them via standard causal attention. Only the projection trains; both base models are frozen. The one firm methodological lesson is that *offline KL convergence is necessary but not sufficient*—v1 converged and still lost—so adapter-style fast/slow coupling should be validated at deployment, not by offline KL alone.

4 Results

4.1 Headline: the Latent Bridge is a safe-or-better drop-in for the Text Bridge

Game	best F (decoder)	best T (decoder)	best L (decoder)	Δ_{L-T} (%, p)	winner
MsPacman	273 (τ1.0)	401 (τ0.5)	628 (greedy)	+57%, p=0.01	**L**
RoadRunner	0 (greedy)	475 (greedy)	608 (greedy)	+28%, p=0.00	**L**
River Raid	994 (greedy)	639 (τ0.5)	566 (τ0.7)	-11%, p=0.35	tie
Seaquest	57 (τ1.0)	143 (τ1.0)	125 (τ0.7)	-13%, p=0.15	tie
Q*bert	65 (τ1.0)	185 (τ1.5)	146 (τ1.0)	-21%, p=0.50	tie
Enduro	4 (τ0.5)	3 (τ0.5)	2 (greedy)	-33%, p=0.39	tie
SpaceInvaders	135 (τ1.0)	162 (τ1.0)	142 (τ1.0)	-12%, p=0.62	tie

Table 1. Headline: best-achievable F/T/L per game, each channel at its own action decoder selected on held-out seeds (leave-one-seed-out; $n = 12$; modal decoder in parentheses). The Latent Bridge is *never significantly worse* than the Text Bridge and significantly *better* on 2 of 7 games (MsPacman, RoadRunner); the other five are statistical ties. Significance is by Mann–Whitney U, appropriate because the best-achievable cells are non-normal (MsPacman's L is bimodal); Welch's t agrees on RoadRunner and is borderline on MsPacman ($p = 0.06$). Reported Stage-A variant per game (§2); both variants in Appendix A.

We compare each channel at its own best action decoder, selected on held-out seeds—the fair comparison, since the decoder is a deployment hyperparameter one tunes, and a single fixed decoder is unfair to both channels (we justify this and contrast it with the rosier fixed-greedy number in §4.4). Under this protocol (Figure 1, Table 1) the Latent Bridge is **never significantly worse** than the Text Bridge and **significantly better on 2 of 7 games**: MsPacman (+57%) and RoadRunner (+28%, the cleanest case, Figure 4). The other five games are statistical ties—no significant difference either way. So at matched architecture the Latent Bridge is a safe-or-better drop-in for the Text Bridge wherever the slow model is worth coupling at all.

The bridges do not always beat Fast-Only. On River Raid and SpaceInvaders, Fast-Only outscores both bridges. The Latent Bridge improves over the Text Bridge *at matched architecture*; whether the slow–fast architecture beats Fast-Only at all is a separate, game-dependent question—and the predictor of §7 says when.

4.2 RoadRunner: the cleanest demonstration

RoadRunner is the cleanest qualitative demonstration (Figure 4): under bare Stage A the action head cannot score at all (its most-confident move is NOOP, even as the Coyote closes), and the slow model's rich joint-state emission (Coyote Δ_x, obstacle and pellet coordinates) unlocks a scoring policy—the Latent Bridge ahead of the Text Bridge. Two caveats keep this honest. The zero-score floor is specific to the *bare* head—a robust head scores well and all three strategies tie (§5)—so this shows the bridge rescuing a brittle baseline, not a fundamental planning limit; and because the baseline is exactly zero, the absolute magnitude is run-to-run-unstable, though the $L > T$ direction is robust (Appendix K). Direct inspection (§6.4) shows the Latent Bridge

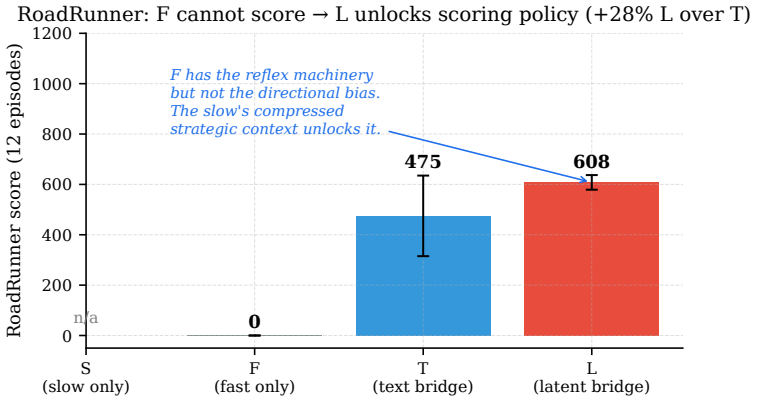

Figure 4. RoadRunner: F cannot score; L unlocks the policy. Under bare Stage A, the action head's most-confident output is NOOP even when the Coyote is closing. The slow's guidance ("head right") breaks the local maximum. $L = 608$ vs $T = 475$, +28%.

encodes mostly game identity plus a thin per-emission strategic residual—non-lexical but game-conditioned.

4.3 Greedy decoding flips low-variance cells—in both directions

Some cells are deterministic under greedy decoding: with a small action space and a confident head, all 12 episodes follow the same trajectory, so the cell has zero variance and the Latent-vs-Text comparison reduces to comparing two integers. On such cells the greedy sign is an artifact—and it cuts *both ways*: greedy hands the Text Bridge a lead on Q*bert and the Latent Bridge a lead on Seaquest, and *both* reverse to ties once actions are sampled (full per-temperature numbers in Appendix H, J).

Why. The Latent Bridge's per-emission variation is tiny (3–8%, §6.4)—below the threshold to flip a greedy argmax, so every episode repeats, but enough to move a *sampled* action distribution. Which channel sampling then favors is game-specific, not a law. The takeaway is that greedy determinism makes the latent-vs-text *sign* unreliable on these zero-variance cells—which is exactly why the headline tunes the decoder per channel rather than trusting a single fixed one (§4.4).

4.4 Why best-achievable, not a fixed decoder: greedy would mislead

Why tune the decoder per channel rather than fix one? Because a single fixed decoder is unfair to both channels at once—they peak at *different* decoders (the Latent Bridge at greedy or high-τ, the Text Bridge at $\tau \approx 0.5$). A fixed *greedy* decoder in particular would tell a rosier story: the Latent Bridge would appear to win *four* games rather than two. But those two extra wins are greedy-specific. They vanish at every fixed sampling temperature (full sweep, Appendix K)—the Latent Bridge's 3–8% per-emission variance (§6.4) nudges the deterministic argmax but is swamped the moment the decoder samples—and on zero-variance cells the greedy sign is an artifact that flips *either* way (it inflates the Text Bridge on Q*bert and the Latent Bridge on Seaquest; §4.3). The held-out selection is also unbiased rather than optimistic: it can land *below* a channel's in-sample greedy score, so the reported +57% on MsPacman is

conservative, not inflated.

Held-out selection. The fair comparison gives each channel its own best decoder, chosen the honest way: tune on held-in seeds, report on held-out. With 3 ALE seeds (4 episodes each) we use leave-one-seed-out—for each held-out seed, pick the decoder with the best mean on the other two, then score the held-out seed under it—and pool the three folds (details in Appendix L). This produces the headline of §4.1 (Table 1): it shrinks the claim from four greedy wins to two decoder-robust ones, but makes it honest. The task-level predictor (§7) is unaffected—it compares each bridge to Fast-Only, and the best bridge beats the best Fast-Only exactly where slow reasoning helps ($T > F$).

4.5 Using both channels at once hurts: the channels interfere, not complement

A natural follow-up to "the Latent and Text channels prefer different decoders and win on different games" is: *use both at once.* We added a combined strategy B that puts the slow model's text suffix in the prompt *and* prepends its latent tokens in the same forward pass, and ran it across all 7 games at the same per-game decoder grid, with the same held-out best-achievable selection (Appendix L). Table 2 compares best-B to the best *single* channel per game.

Game	best single (T or L)	best B (T+L)	$\Delta_{B-\text{single}}$ (%)	p	effect
MsPacman	628 (L)	319	-49%	0.00	**interferes**
RoadRunner	608 (L)	25	-96%	0.00	**interferes**
River Raid	639 (T)	452	-29%	0.01	**interferes**
Seaquest	143 (T)	138	-3%	0.71	neutral
Q*bert	185 (T)	125	-32%	0.26	neutral
Enduro	3 (T)	4	+33%	0.18	neutral
SpaceInvaders	162 (T)	142	-12%	0.62	neutral

Table 2. Combining both channels (best B) vs. the best single channel, held-out best-achievable, $n = 12$. Combining is significantly *worse* on 3 of 7 games, never significantly better, and on RoadRunner it is catastrophic (-96%, near the $F = 0$ floor). The channels *interfere*.

The naive "more context is better" intuition fails. Combining the channels never significantly beats the better single channel and significantly *hurts* on 3 of 7 games (catastrophically on RoadRunner; Table 2). The damage concentrates exactly where one channel is strong on its own—both the latent-dominated and the text-dominated games degrade when the second signal is added. We attribute this to the frozen action head, trained on a *single* conditioning signal at a time (Stage A sees a bare prompt or a suffix; Stage C distills the Latent Bridge toward the Text Bridge): handed both at once, it produces a worse policy. **The design rule is therefore: couple via exactly one channel.** The predictor (§7) says *whether* to couple ($T > F$); if so, pick a single channel, with the Latent Bridge the safe-or-better default.

Figure 5. One Stage A retraining recovers two distinct collapse modes. SpaceInvaders (left): bare collapses both bridges to zero; robust lifts them off the floor (though still below Fast-Only). River Raid (right): bare is a near-tie far below Fast-Only; robust leaves the Text Bridge flat but lifts the Latent Bridge well above it under greedy decoding—a margin that itself becomes a tie once the decoder is tuned per channel (§4.4), which is why River Raid is a tie in the headline. Greedy scores in Appendix A.

5 Mechanism: Stage A action-head OOD-brittleness

Three games went wrong in their bare-Stage-A sweeps despite cleanly converging Stage C training: SpaceInvaders and Q*bert collapsed to zero, and River Raid fell to a bridge-harmful near-tie far below Fast-Only. The same robust-Stage-A retraining recovers two distinct collapse modes (Figure 5). The collapse is not a bridge-training failure: on SpaceInvaders, three orthogonal interventions (random-policy and expert-policy T-trajectories, and an aggressive "FIRE constantly" slow prompt) all left it intact.

Diagnosis, and a direct measurement. Stage A trains the action head on the *bare* game-state prompt; at deployment, the text suffix and the bridge tokens are both OOD inputs. We measure the effect directly (Appendix G): on bare Stage A, the suffix **flips the action head's argmax on most emissions (70–90%) of every game**—the drift is large and uniform. So the perturbation is real; what differs across games is *what action the drift moves toward*—on some games the new argmax still scores, on others it lands on a bad action and scoring collapses, a property of each game's reward structure rather than of the bridge. The bridge mechanism is fine; the frozen action head's suffix-induced argmax flip is the binding constraint.

The fix. Retrain Stage A at suffix-probability 0.5: half of training batches receive a randomly-chosen slow-style text suffix appended to the prompt. The head learns suffix-invariance.

The fix is not universal. Robust Stage A rescues the games whose bare bridges collapsed (SpaceInvaders, River Raid—dramatically lifting the Latent Bridge on River Raid) but *harms* the games whose bare bridges already worked, destroying the Latent Bridge on MsPacman and Seaquest and erasing the advantage on RoadRunner (where it instead lifts Fast-Only off its zero floor, so all three strategies tie). **Recommendation**: use robust Stage A only when a bare bridge collapses to near zero; otherwise prefer bare. Per-game scores in Appendix A.

Robust Stage A reduces drift exactly where expected. On the non-collapsed games, robust Stage A sharply cuts the suffix-induced argmax drift (e.g. RoadRunner 75% → 12%)—except Seaquest, whose drift *rises*, matching robust Stage A's deployment harm there. On the collapsed games it lowers the KL but only partly reduces the drift: the head grows more confident under the suffix without always landing on a recoverable action. Full per-game probe in Appendix G.

6 Channel ablations

6.1 The channel is not bandwidth-limited

Sweeping the latent token count $N \in \{4, 8, 16\}$ on MsPacman does not behave like a bandwidth bottleneck. Trained and deployed at a matched N, the Latent Bridge peaks at $N = 8$ (628, vs 296/259 at $N = 4/16$); but *deploying* a bridge trained at $N = 8$ with more positions ($N = 16$) scores higher still (720), exploiting positions it was never trained for. The gating quantity is thus not how many tokens the channel carries—consistent with the low per-emission variance of §6.4—so we do not advance a "text is bandwidth-limited" account.

6.2 A longer text suffix does not close the gap—it widens it

The simplest objection to the Latent Bridge's advantage is "the Text Bridge just needs more text." We tested it directly: on MsPacman and RoadRunner we re-ran the Text Bridge with a rolling window of the last $w \in \{1, 2, 3\}$ slow emissions concatenated into the suffix, holding the model, checkpoint, seeds, and episode count fixed (Figure 6; full numbers in Appendix I). Concatenating older emissions *hurts*: stale emissions describe game state that no longer holds, and the action head cannot disambiguate their time ordering. The decline is gentle on MsPacman (slower-changing pellet strategy) and total on RoadRunner (moment-to-moment obstacle dodging), where every episode scores zero by $w = 3$. Because the Latent Bridge transmits only the most-recent emission, its score is unchanged by w, so the gap only widens. To compete, the Text Bridge would need explicit time-tagging or a learned attention over emissions.

6.3 The bridge tokens carry information beyond "prepended slots"

A negative-control objection to the Latent Bridge: maybe the trained tokens do nothing, and the LLM merely benefits from 8 extra prepended positions to compute over. We tested it on MsPacman by replacing the trained projection's output with 8 zero vectors or 8 random vectors at the trained per-position norm, holding everything else fixed (Figure 7; significance tests in Appendix M). Both effects are real and separable: the empty slots alone lift the policy ≈40% over Fast-Only (a pause-token-like effect [8]), and the *trained* content adds the remaining ≈60%—the part that distinguishes the Latent Bridge from a generic "more compute per tick" baseline. This single-game split is not universal: the all-games version of this control (§7.3) finds the learned fraction ranges from ≈100% (RoadRunner) to negative (River Raid), with its sign predicted by whether slow reasoning helps the task ($T > F$).

6.4 What the bridge encodes: not text, mostly game identity

We project the trained bridge tokens of each game (with cached slow residuals from seed-0 trajectories, 8 emissions per game) into the fast LLM's input embedding space and compare

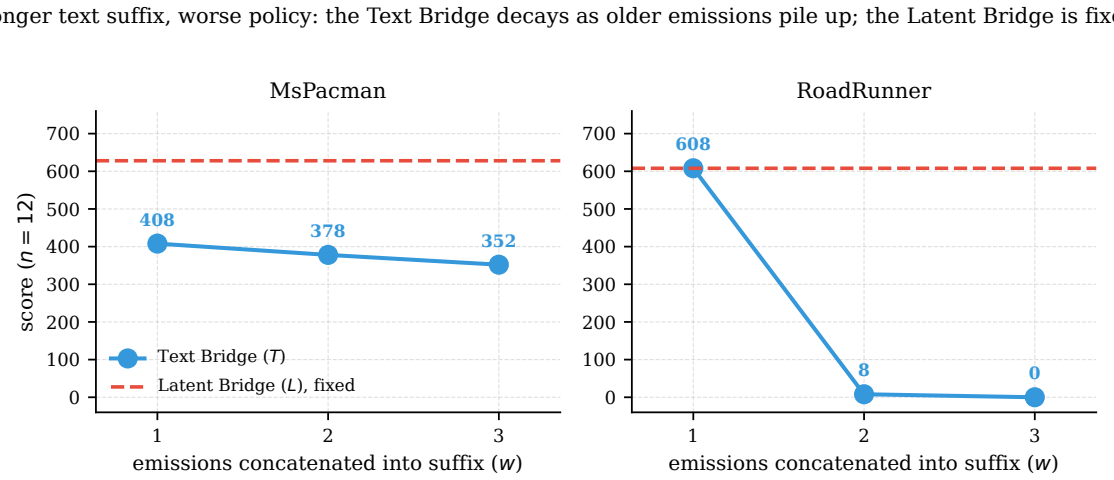

Figure 6. A longer text suffix widens the gap. The Text Bridge's score as a function of how many recent slow emissions are concatenated into the suffix; the Latent Bridge (latest emission only) is the fixed dashed line. (RoadRunner is run-to-run-variable, §4; shown is one internally consistent run, where the within-run $w = 1$ baseline starts at 608 and collapses—the trend, not the headline level.)

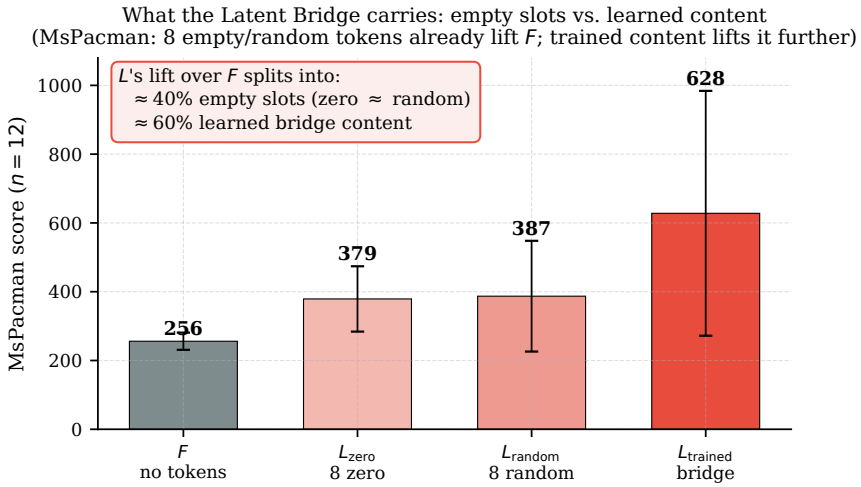

Figure 7. The latent's lift over Fast-Only splits into architectural "slots" and learned content. On MsPacman, 8 zero or random prepended tokens already lift Fast-Only (≈40% of the gain); the trained bridge adds the rest (≈60%). Zero and random are statistically indistinguishable, so the slot effect is content-free; trained beats both (Appendix M).

them directly to the fast LLM's 151,748 vocab embeddings.

The bridge is non-lexical. Bridge tokens have L2 norm ∼64 (constrained by the output LayerNorm); vocab embeddings have norm ∼1.45 (p99 1.92). The cosine similarity of any bridge token to its nearest vocab embedding is ≤ 0.09 across all 7 games. Plug-in nearest-neighbor decoding returns visually random tokens (rare Unicode characters, code-snippet fragments) at sims indistinguishable from the 99.9th-percentile sim across the whole vocabulary. **The latent channel does not encode in the vocabulary basis**—it lives in a region of $\mathbb{R}^{4096}$ that the frozen LLM has learned to attend to but that no text token occupies. This is consistent

with what LLaVA's vision projection produces (also non-lexical), and is the right answer to the question "can we just read the latents as text?"—we cannot.

The bridge is game-conditioned. Bridge tokens of the *same* game cluster tightly (mean pairwise cosine **+0.80 to +0.89**) while *different* games are nearly orthogonal (**+0.04 to +0.25**), with related games (the spatial-2D set) closer than unrelated ones like forward-driving Enduro. Full 7×7 matrix in Appendix F.

The bridge is weakly per-emission-conditioned. Cosine between the *mean bridge token* of one emission vs another emission *within the same game*: +0.92 to +0.97 (std 0.02–0.03). The bridge varies only ∼3–8% between consecutive emissions of the same game. This is a surprisingly low per-emission variance and explains the deterministic-trajectory outcomes (e.g., Q*bert's std=0 cells): if every emission produces near-identical bridge tokens, the fast model picks near-identical actions, so 12 episodes converge on the same trajectory under greedy decoding.

Implication: used capacity $\ll$ nominal 8×4096. The latent channel's nominal capacity is enormous, but its *used* capacity is small: most of its variance encodes the (episode-fixed) game identity, and only a thin fraction encodes per-emission strategic state. So the latent's measured advantage is a *partial utilization*, not a capacity ceiling. This is why what gates the bridge is whether the slow channel carries a useful signal at all (§7), not how many bits it could carry.

7 When the bridge helps: a behavioral predictor

The headline tally (§4) says *where* the Latent Bridge wins, and the ablations (§6) say the reason is not bandwidth; this section asks *when* the bridge pays off. Across every domain we evaluated, a single behavioral variable—whether slow reasoning helps the task at all ($T > F$)—predicts it. We first extend the testbed beyond Atari (§7.1), then state the predictor (§7.2) and its mechanistic corroboration (§7.3), and close with an axis that does *not* predict (§7.4).

7.1 Non-Atari test: a driving domain

To probe whether the bridge generalizes beyond Atari, we ported the full pipeline to **MetaDrive** [13], a real-time driving simulator with the same fast/slow premise (a ∼10 Hz reactive control loop; a ∼1.5 s reasoning step that cannot sit in it). We use a top-down rendered observation for the fast model and the same structured state decoder (speed, lane offset, heading, upcoming-turn cue, neighbours) for the slow model. This is, to our knowledge, the first non-game test of the Latent Bridge; the integration runs end-to-end on one GPU (Appendix P).

The bridge does not transfer to driving—and the controls explain why. On the default lane-keeping map the Latent Bridge ties Fast-Only and the Text Bridge collapses; the bridge-replacement control (Appendix M, method) shows the Latent Bridge is *inert* (zeroing or randomizing the bridge tokens leaves the score unchanged, $L \approx L_{\text{zero}} \approx L_{\text{random}} \approx F$). Crucially, this is *not* the Stage A OOD artifact of §5: it persists on the robust head, under sampling, and after retraining Stage C on expert-driven trajectories with a healthy ($T \approx F$) teacher.

It is also not "the task had no slow component." We rebuilt MetaDrive as a *planning-heavy* route (straights punctuated by intersections, with off-route termination) where survival genuinely requires deliberate turn decisions—the navigation expert beats a naive straight-driving policy by +118 reward, and the slow model receives explicit turn cues. Even here (Figure 8, right) the slow model does not beat Fast-Only—it ties under greedy and actively *hurts* under sampling, with the Latent Bridge tracking the Text Bridge down (numbers in Appendix P). Driving is dominated by a tight visuomotor loop the fast model already handles; the slow model's latency-bound, frame-grounded reasoning does not improve on it, unlike Atari planning games whose slower dynamics reward deliberation.

7.2 The predictor: the Latent Bridge helps iff the slow model helps

This tracking is partly built in by construction: Stage C trains the Latent Bridge to imitate the Text Bridge's action distribution (minimizing $\mathrm{KL}(\pi_L \,\|\, \pi_T)$), so the Latent Bridge can at best convey what the Text Bridge already conveys—it can only help where the Text Bridge helps. The data bear this out. Pooling all eight game-evals (7 Atari + MetaDrive; Figure 8, left), the Latent Bridge's benefit over Fast-Only ($L-F$) tracks the Text Bridge's benefit ($T-F$) at **Pearson** $r = 0.93$ (bootstrap 95 % CI $[0.81, 1.0]$). The relationship is not an artifact of variant selection: it holds at $r = 0.96$ over all 16 evaluated cells—the 14 bare/robust Atari cells plus a SpaceInvaders expert-data cell and MetaDrive, with no per-game selection (§2, Appendix P). The bridge pays off exactly when slow reasoning beats reaction on the task ($T > F$: RoadRunner, MsPacman, Seaquest, Q*bert; Enduro sits at floor, with $T-F$ a few points either side of zero depending on variant) and is inert or harmful when it does not (River Raid, SpaceInvaders, MetaDrive). This reframes "does a Latent Bridge help?" as a property of the *task* (is the bottleneck deliberation or perception–action?), not of the channel—and makes MetaDrive a clean controlled negative rather than a failed port. The bridge-replacement control is the diagnostic that separates a genuinely informative latent (MsPacman: $L_{\text{trained}} \gg L_{\text{random}}$) from an inert one (MetaDrive).

7.3 Corroboration: learned content tracks the predictor

The bridge-replacement control—replace the trained bridge tokens with zeros or with random vectors at matched norm, and check whether the score drops (§6.3 analyzes the MsPacman case in depth)—doubles as a per-game diagnostic of whether the Latent Bridge carries learned, behavior-relevant content. We ran it on all 7 Atari games; "learned" means L_{trained} exceeds $\max(L_{\text{zero}}, L_{\text{random}})$ by $>10\%$. Cells use the bare Stage A head under greedy decoding (Q*bert: robust head under $\tau = 1$ sampling, its canonical decoder); $T-F$ is the matching bare-head value, and the run is an independent re-run whose repeated cells agree with the headline within run-to-run noise (full numbers and caveats in Appendix M).

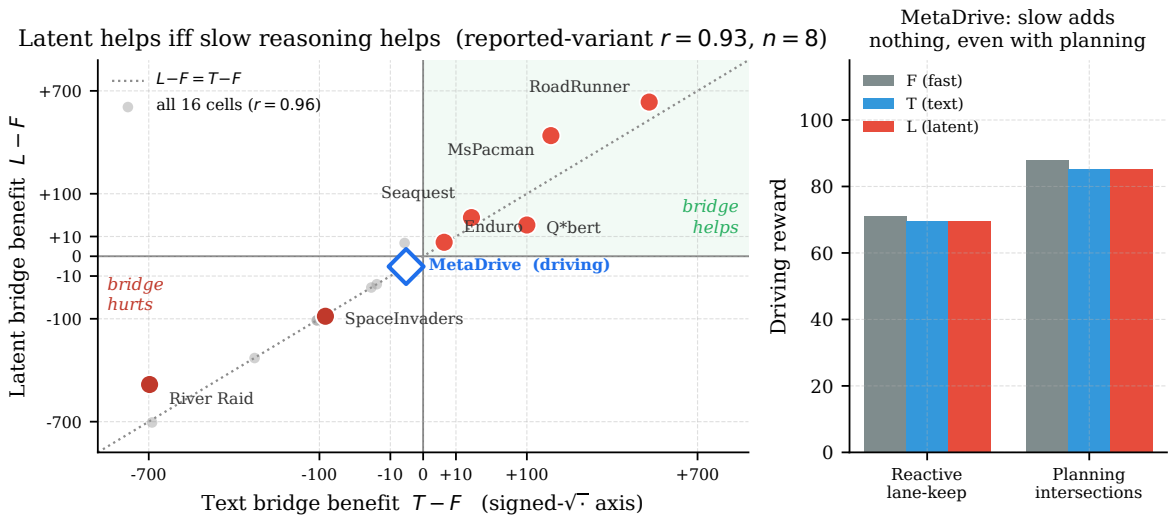

Figure 8. The Latent Bridge helps if and only if the slow model helps the task. *Left:* per-game Latent benefit $L-F$ vs. Text benefit $T-F$ across 7 Atari games and MetaDrive, on a signed-$\sqrt{\cdot}$ axis. Bold points are the reported-variant cell per game (Pearson $r = 0.93$, $n = 8$); faint grey points are all 16 evaluated cells ($r = 0.96$), so the relationship is not an artifact of variant selection (robustness stats in Appendix P). The bridge helps only in the upper-right quadrant ($T > F$ and $L > F$); MetaDrive (blue diamond) sits at the origin. *Right:* in both MetaDrive regimes—reactive lane-keeping and planning-heavy intersections—slow reasoning never exceeds Fast-Only, even when the task demands route planning.

Game	L_{train}	L_{zero}	L_{rand}	verdict	$T-F$
RoadRunner	**608**	0	8	learned	+475
Seaquest	**100**	28	5	learned	+22
MsPacman	**666**	408	410	learned	+152
Enduro	7.8	1.4	4.7	trend only (at floor)	−3
Q*bert	123	63	117	≈random	+100
SpaceInvaders	0	148	90	*harmful*	−105
River Raid	360	1013	1003	*harmful*	−683

The verdict tracks the predictor. The trained latent is genuinely learned exactly on the games where slow reasoning helps ($T > F$: RoadRunner, Seaquest, MsPacman)—on RoadRunner the controls drop to the floor, so the Latent Bridge there is almost entirely learned content. Where slow reasoning does not help ($T \leq F$: River Raid, SpaceInvaders), the trained latent is *harmful*: zeroing or randomizing it scores *better*—the same inert/harmful pattern as MetaDrive (§7.1). The two off-diagonal cases are benign: Enduro's scores are too small for the comparison to be meaningful, and Q*bert—the decoder-fragile game (§4.3)—shows trained ≈ random under sampling, consistent with its thin per-emission signal. The latent helps where, and only where, it has carried real content from a slow channel that itself beats Fast-Only.

7.4 What does not predict: emission statistics

We also attempted a quantitative axis. We hypothesized lexical diversity of slow emissions (unique whitespace-split tokens per emission) would predict the sign of $L-T$. Across our 7 games (Figure 9), $r = -0.08$ (slope -0.01 per diversity-unit; not significant). Six alternative

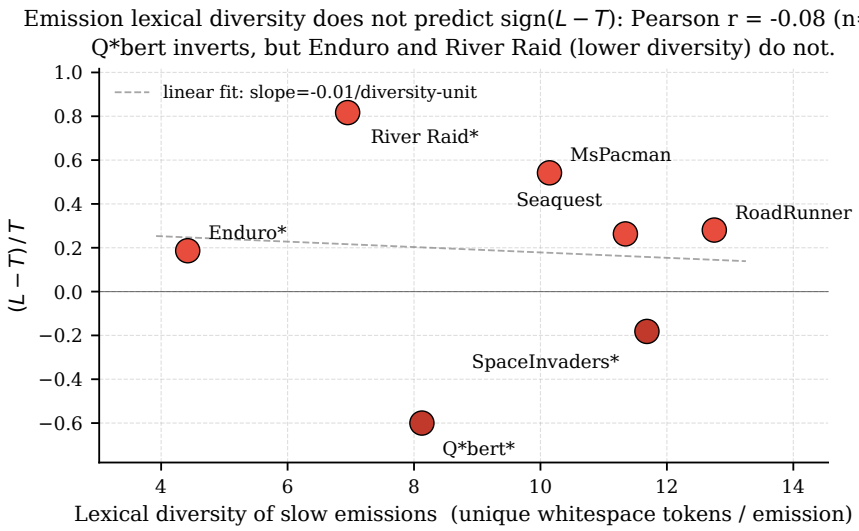

Figure 9. Emission statistics do not predict the sign of $L - T$ ($n = 7$). x = mean unique whitespace tokens per slow emission (seed-0 T-trajectory). $y = (L - T)/T$. Q*bert is the only $y < 0$ point with a large T, but Enduro and River Raid (lower diversity) have $y > 0$, and SpaceInvaders (high diversity) is mildly $y < 0$. Pearson $r = -0.08$. The dashed line is the linear fit and is shown only to make the lack of structure visible.

features (gzip ratio, numeric density, distinct numbers, token counts) all top out at $|r| \leq 0.25$. We retain the figure as a negative result: emission *statistics* do not predict the sign of $L-T$; the behavioral variable $T-F$ (§7.2) does predict $L-F$.

8 Discussion

8.1 Two sharpened claims

Tuned per channel, the Latent Bridge is a safe-or-better drop-in for the Text Bridge. Because the action decoder is a deployment hyperparameter, we give each channel its own best decoder on held-out seeds (§4.4, Appendix L): there the Latent Bridge is *never significantly worse* than the Text Bridge and significantly *better* on 2 of 7 games (MsPacman +57%, RoadRunner +28%), with 5 ties. A single fixed greedy decoder would inflate this to 4 apparent wins, but that edge is decoder-specific—it vanishes under sampling and flips on zero-variance cells (§4.3)—so the decoder-robust claim is the honest one.

The Latent Bridge helps iff the slow model helps the task. Pooling 7 Atari games and a driving domain, $L-F$ tracks $T-F$ at Pearson $r = 0.93$ ($r = 0.96$ over all 16 evaluated cells; Figure 8). Because Stage C distills the Latent Bridge toward the Text Bridge, the Latent Bridge's ceiling is the Text Bridge: the bridge is worthwhile exactly when slow reasoning beats reaction ($T > F$), and inert/harmful otherwise. MetaDrive is the controlled negative ($T \leq F$ even in a planning-heavy regime, bridge-replacement control confirming an inert Latent Bridge). Whether a bridge pays off is thus a property of the task's bottleneck (deliberation vs. perception–action), not of the channel.

8.2 Limitations

- **Small n.** 12 episodes per cell. Several cells have zero per-episode variance (greedy decoding + deterministic policy + small action space); on those, parametric statistics are not meaningful and we report MWU or just note the deterministic outcome.
- **Per-game variant selection.** The headline reports, per game, the Stage A variant with the higher deployment L among variants whose T has not collapsed (§2); this is a per-game choice. We report both variants for every game in Appendix A, and show the cross-game predictor holds with no selection ($r = 0.96$ over all 16 evaluated cells).
- **The token-count (N) ablation is three points.** The deploy-only result (a bridge trained at $N=8$ scoring best deployed at $N=16$) is noted in §6.1.
- **Per-emission information content is not measured directly.** We show in §6.4 that the bridge is strongly game-conditioned but only weakly per-emission-conditioned; a quantitative bound on the Latent Bridge's per-emission information content is the natural next step.
- **Beyond Atari: one driving domain, a controlled negative.** We ported the pipeline to MetaDrive (§7.1, Appendix P); the bridge does not help there, and the bridge-replacement control shows why (the Latent Bridge is inert because the slow model's guidance never beats Fast-Only, $T \leq F$). This is consistent with the $(T-F, L-F)$ predictor ($r = 0.93$) but is a single non-game domain; we still have not tested the motivating target setting—real-time computer-use and game agents on phone and desktop, with their much larger action and observation spaces.

Two further negative results reinforce the picture: a longer rolling text suffix *widens* rather than closes the gap (§6.2), and swapping in a 30B-A3B MoE slow model does not grow the Latent-over-Text gap on the one game tested (Appendix N).

8.3 Implications

For deployments where slow reasoning does not improve on fast reaction ($T \leq F$, MetaDrive-like), neither bridge pays for itself, and the Text Bridge is the cheaper way to find that out. For deployments where slow reasoning does help ($T > F$), the Latent Bridge is a safe-or-better drop-in for the Text Bridge: tuned per channel, it is never significantly worse than the Text Bridge and wins by up to +57% where it wins significantly (MsPacman, RoadRunner; ties on the rest, §4.4). Two caveats apply: the advantage comes through better greedy action selection (it does not survive a *fixed* stochastic decoder), and on the one game with an $F=0$ baseline (RoadRunner) the absolute magnitude is run-to-run-sensitive. The architectural fix is not novel—LLaVA's pattern applied to a non-vision coupling problem—but the empirical demonstration at this scale, with an honest decoder-sensitivity analysis and a mechanism for the collapses, is new to our knowledge.

9 Related Work

Real-time and computer-use agents. Closed streaming systems—Gemini Live [7], OpenAI Realtime [15]—fold perception and shallow thinking into one model. Closest in spirit is Thinking Machines Lab's *Interaction Models* [21], which make the fast/slow split explicit—a live interaction model plus an asynchronous background reasoner—but couple them through *na-*

tively shared context (the background model receives the full conversation), a richer mechanism than—though akin in spirit to—the text suffix our T channel feeds a frozen reader, and within a single closed, jointly-trained system rather than two independent frozen models. We neither use nor compare against these directly; we note only that open-source VLMs expose no equivalent fast/slow split, which is the gap our coupling fills, and that we test a *learned latent* channel as the alternative to their text/context coupling. Computer-use and GUI agents (operating a phone or desktop from pixels) inherit exactly this tension between a tight perception–action loop and goal-level planning. SayCan [1] and Inner Monologue [10] couple LLM planners to skill policies via natural-language sub-goals; RT-2 [5] unifies them with a discrete-token action head. Our setting holds the architecture fixed and varies only the *channel* between two frozen general-purpose multimodal LLMs at matched scale.

Multimodal coupling. LLaVA [14], BLIP-2 [12], Flamingo [2], MiniCPM-o [16] couple vision and language by prepending projected visual tokens to an LLM's input. BLIP-2 ablates query-token count, the closest prior analog to our N sweep. Our v2 transfers this pattern to language-language fast/slow coupling.

Continuous chain-of-thought. COCONUT [9], Pause Tokens [8], Implicit CoT [6], Quiet-STaR [22] all reason in latent space within a single model. Our work is the cross-model analog: two frozen models coupled by a learned latent channel.

Hierarchical / cascades. Speculative decoding [11] couples draft+verifier within one decoding loop; Mixture-of-Depths [19] adapts compute per token. These optimize a single forward pass; we decouple two models in wall-clock and pass the slower's deliberation forward as a learned signal.

Position. To our knowledge this is the first reported head-to-head text-vs-latent comparison between two frozen general-purpose multimodal LLMs at matched scale, with deployment-time evaluation (not just offline KL) and a controlled negative domain (MetaDrive, where neither bridge helps) that bounds the claim.

10 Conclusion

We coupled two frozen, matched-scale multimodal LLMs (9 B fast, 8 B slow) through a learned continuous Latent Bridge and compared it head-to-head with the Text Bridge those fast/slow systems normally use. Tuned per channel on held-out seeds—the fair test, since the decoder is a deployment hyperparameter—the Latent Bridge is a safe-or-better drop-in: never significantly worse than the Text Bridge and significantly better on 2 of 7 games (§4.4). Three results sharpen the design picture: feeding *both* channels to the frozen head does not combine their strengths but *interferes* (RoadRunner −96%), so couple via exactly one (§4.5); most "bridge collapsed" cells are Stage A action-head OOD-brittleness, not bridge failures, and a targeted retraining recovers them (§5); and the obvious objections do not hold—a longer text suffix *widens* the gap (§6.2) and zero/random latents cost most of the advantage where it wins (§6.3).

Rather than a "text is bandwidth-limited" story—which our own ablations do not support (§6.1, Appendix N)—we land on a sharper claim: **the Latent Bridge helps if and only if slow**

reasoning helps the task ($L-F$ tracks $T-F$ at $r = 0.93$; MetaDrive is the controlled negative where the slow model never beats Fast-Only and the Latent Bridge is correspondingly inert, §7).

The cleanest follow-ups are (i) an emission-discrimination auxiliary loss during Stage C to push more per-tick state into the bridge—the trained Latent Bridge is non-lexical and strongly game-conditioned but only weakly per-emission-conditioned, so its used capacity is far below its nominal 8×4096 (§6.4)—and (ii) scaling the comparison to the target setting that motivates it: real-time computer-use and game agents on phone and desktop, where the same fast-react/slow-plan tension holds but the action and observation spaces are far larger than Atari.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *CoRL*, 2022.

[2] Jean-Baptiste Alayrac et al. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022.

[3] Ankesh Anand, Evan Racah, Sherjil Ozair, Yoshua Bengio, Marc-Alexandre Côté, and R Devon Hjelm. AtariARI: Atari annotated RAM interface, 2019. https://github.com/mila-iqia/atari-representation-learning.

[4] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

[5] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *CoRL*, 2023.

[6] Yuntian Deng, Yejin Choi, and Stuart Shieber. From explicit CoT to implicit CoT: Learning to internalize CoT step by step, 2024. arXiv:2405.14838.

[7] Google. Gemini Live multimodal real-time api, 2024. https://ai.google.dev/gemini-api/docs/live.

[8] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *ICLR*, 2024.

[9] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space, 2024. arXiv:2412.06769.

[10] Wenlong Huang, Fei Xia, Ted Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In *CoRL*, 2022.

[11] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[12] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, 2023.

[13] Quanyi Li, Zhenghao Peng, Lan Feng, Qihang Zhang, Zhenghai Xue, and Bolei Zhou. MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(3):3461–3475, 2023.

[14] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[15] OpenAI. OpenAI Realtime API, 2024. https://platform.openai.com/docs/guides/realtime.

[16] OpenBMB Team. MiniCPM-o 4.5: An omni-modal large language model, 2025. https://huggingface.co/openbmb/MiniCPM-o-4_5.

[17] Qwen Team. Qwen3-VL-8B-Thinking, 2025. https://huggingface.co/Qwen/Qwen3-VL-8B-Thinking.

[18] Antonin Raffin. Stable-Baselines3 atari zoo, 2021. https://huggingface.co/sb3.

[19] David Raposo, Sam Ritter, Blake Richards, et al. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024. arXiv:2404.02258.

[20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.

[21] Thinking Machines Lab. Interaction models: A scalable approach to human–AI collaboration, 2026. https://thinkingmachines.ai/blog/interaction-models/.

[22] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D Goodman. Quiet-STaR: Language models can teach themselves to think before speaking. In *COLM*, 2024.

A Full results table

Both bare-Stage-A and robust-Stage-A reported for every game, $n = 12$ per cell.

Game (variant)	F	T	L	Δ_{L-T} (%, p)
MsPacman	256 ± 25	408 ± 92	628 ± 356	+54 % (p=0.06)
MsPacman (robust SA)	325 ± 72	61 ± 3	60 ± 0	-1 % (p=0.36†)
Seaquest	42 ± 20	63 ± 12	80 ± 0	+26 % (p<.001†)
Seaquest (robust SA)	20 ± 0	0 ± 0	0 ± 0	–
RoadRunner	0 ± 0	475 ± 160	608 ± 29	+28 % (p=.015)
RoadRunner (robust SA)	958 ± 67	1000 ± 0	925 ± 45	-8 % (p<.001†)
River Raid	1067 ± 88	383 ± 60	360 ± 0	-6 % (p=0.01†)
River Raid (robust SA)	1032 ± 20	337 ± 81	612 ± 310	+82 % (p=0.01)
Enduro	3 ± 3	0 ± 0	8 ± 9	–
Enduro (robust SA)	1 ± 1	5 ± 6	6 ± 3	+19 % (p=0.63)
Q*bert	25 ± 0	0 ± 0	0 ± 0	–
Q*bert (robust SA)	25 ± 0	125 ± 0	50 ± 0	-60 % (p<.001†)
SpaceInvaders	105 ± 0	0 ± 0	0 ± 0	–
SpaceInvaders (robust SA)	107 ± 62	18 ± 19	15 ± 0	-18 % (p=0.27†)
Pong	-21 $\pm$ 0	-21 $\pm$ 0	-21 $\pm$ 0	+0 % (p=1.00†)

Reading the table. Welch's t becomes undefined or unreliable when either cell has zero variance; in those cases we report Mann–Whitney U (marked †). On cells where *both* sides have zero variance (e.g., Q*bert/robust $T = 125 \pm 0$, $L = 50 \pm 0$), the test reduces to comparing two integers; the p-value is mechanical, not a sampling statement.

B Statistics methodology

Variance sources. Each cell is 3 seeds $\times$ 4 episodes per seed $= 12$ episodes. Seeds vary the ALE initial state. Action selection is greedy (argmax over logits). For games with small action spaces and policies that consistently pick the same action sequence, every episode within a seed often hits the same trajectory; hence many cells have $\sigma = 0$. This is not a sampling artifact—it is the policy being deterministic given the env seed.

When we trust which test.

- Both cells have $\sigma > 0$ and roughly normal: **Welch's** t.
- One cell has $\sigma = 0$, the other has support: **Mann–Whitney** U.
- Both cells have $\sigma = 0$: **we report the comparison but do not claim a** p**-value** (the test is deterministic-trajectory difference, not a population inference).

Test convention. All Welch's t values use the unbiased sample-variance form, as in SciPy's `ttest_ind` with `equal_var=False`.

Bootstrap CIs. 10^4 resamples for each cell mean.

Cohen's d. Reported only when both cells have $\sigma > 0$. Zero-variance cells produce mechanically large d that we do not interpret.

C Slow emission samples

RoadRunner.

> *"Got it, let's break this down. The current state: Road Runner is at (3,0), Coyote is at (0,0) with a Δ_x of +3, so Coyote is closing in. The nearest birdseed is at $x = 159$, which is ahead. There are obstacles at (170,16) and (170,0), so trucks/landmines are in the way. …"*

Q*bert.

> *"Got it, let's analyze the Q*bert state. The player is at (0,0), Coily is at (65,0) which is on the right edge. The purple and green balls are at $y = 148$, so they're falling. The goal is to jump tiles to change colors, avoid Coily and balls. First, threat: Coily is on the right edge, so if Q*bert moves…"*

Both samples are ∼300 characters. RoadRunner's encodes simultaneous Δ_x for Coyote, target x for birdseed, two obstacle coordinates, and a directional commitment. Q*bert's encodes (jump-direction, target-color, threat-actor)—a triple. This is the qualitative content that motivated the lexical-diversity hypothesis tested (and rejected) in §7.4; we make no quantitative prediction on top.

D Stage A and Stage C details

Stage A. AdamW, lr 1×10^{-4}, 3 epochs, batch=1 with grad-accum=8. The action head has 18 outputs (full ALE action space) on every game; the random baseline below is $1/k$ for k = the game's ALE *minimal* action set (the only actions the SB3 expert emits, so the val-set classes). Bare val-accuracies on the game-state-only prompt: MsPacman 32% (random 11.1%, $k=9$), Seaquest 24% (5.6%, $k=18$), SpaceInvaders 33% (16.7%, $k=6$), Pong 25% (16.7%, $k=6$), RoadRunner 59% (5.6%, $k=18$), River Raid 31% (5.6%, $k=18$), Enduro 49% (11.1%, $k=9$), Q*bert 34% (16.7%, $k=6$). Robust Stage A is identical except half of training batches receive a synthetic slow-style text suffix appended (suffix-prob=0.5); bare val-acc barely changes (e.g., MsPacman 32→33%, SI 33→30%) but the policy is suffix-invariant.

Stage C. 2-layer MLP projection (LayerNorm, Linear 4096→4096, GELU, Linear 4096→4096, LayerNorm), 33.6 M parameters. AdamW, lr 5×10^{-5}, 1 epoch, batch=1 with grad-accum=4. KL temperature $\tau = 1.0$. Slow residuals extracted at layer 24 of 36 (we did not ablate the layer choice). Default $N = 8$ latent tokens, ∼5K (frame, slow text, slow residuals) tuples per game from Stage B random-policy rollouts.

Text-bridge suffix. The slow's full unmodified post-thinking emission, appended verbatim under a "strategic-guidance" tag; no truncation; median 302 chars across games.

Slow prompt template (excerpt, RoadRunner, paraphrased). The object coordinates fed to the slow model are extracted from the ALE RAM in the style of the Atari Annotated RAM Interface [3]. "Road Runner is at (x_{rr}, y_{rr}), Coyote at (x_c, y_c) with $\Delta_x = +3$. Nearest birdseed at $x = 159$; obstacles at $(170, 16)$ and $(170, 0)$. …" The system prompt instructs the slow to emit 1–3 sentences identifying threat/opportunity and recommended intent—*not* individual actions.

E Latency breakdown

All experiments run on a single NVIDIA RTX Pro 6000 (96 GB). Per-tick wall-clock on this host, with vLLM workers sharing the same GPU (hence contended):

Stage	F (no cache)	F (vrf=4)	L (vrf=4)
Vision tower	70 ms	17.5 ms[a]	17.5 ms
LLM prefill	60 ms	13 ms	18 ms[b]
LLM decode (1 token)	27 ms	2.5 ms	2.5 ms
Per-tick total	∼157 ms	∼33 ms	∼38 ms

[a]amortized over the 4-tick cache window; [b]includes the 8 prepended latent tokens.

The bridge itself adds only ∼5 ms over F. End-to-end "$L \approx 124$ ms" figures from earlier work also include async-callback completion time under GPU contention. Vision-token caching across N ticks (vrf) reduces vision-tower cost by $N\times$ at the cost of perception staleness; score is non-monotonic in vrf (vrf=4: $F = 110$, vrf=15: $F = 140$ on MsPacman), an interaction of staleness with multi-tick action commitments. (These are a separate latency-stress configuration with caching enabled and so do not match the headline MsPacman $F = 256$, which uses per-tick vision.)

F Bridge-token diagnostics (full table)

Backing data for §6.4. Bridge tokens computed by loading the trained Stage C v2 projection for each game and applying it to that game's cached seed-0 slow residuals; up to 8 emissions per game.

Norms. Vocab embeddings: mean 1.453, std 0.359, p1=0.281, p99=1.923. Bridge tokens: norm ≈64 across all 7 games (std $\leq$ 0.01 within a game)—fixed by the output LayerNorm of the projection MLP.

Nearest-vocab cosine, averaged over the 8 bridge positions of one emission:

Game	avg max cosine to vocab	avg p99.9 cosine to vocab
MsPacman	0.076	0.053
RoadRunner	0.081	0.058
Q*bert	0.087	0.058
Seaquest	0.067	0.044
River Raid	0.074	0.050
SpaceInvaders	0.070	0.047
Enduro	0.063	0.034

For reference, two random vocab embeddings have cosine ≈0.04. Every bridge token is essentially at that random-vocab background. The "top-k tokens" from a plug-in nearest-neighbor decode are therefore the maximum of a uniform-low distribution, not real semantic matches.

Cross-game cosine (mean pairwise bridge similarity over $E \times N$ tokens):

	MsPac	RR	Q*b	Seaq	RivR	SI	Endu
MsPacman	+0.87	+0.18	+0.13	+0.21	+0.10	+0.25	+0.04
RoadRunner	+0.18	+0.81	+0.13	+0.21	+0.09	+0.15	+0.10
Q*bert	+0.13	+0.13	+0.81	+0.18	+0.15	+0.14	+0.04
Seaquest	+0.21	+0.21	+0.18	+0.80	+0.11	+0.23	+0.09
River Raid	+0.10	+0.09	+0.15	+0.11	+0.89	+0.14	+0.06
SpaceInvaders	+0.25	+0.15	+0.14	+0.23	+0.14	+0.85	+0.06
Enduro	+0.04	+0.10	+0.04	+0.09	+0.06	+0.06	+0.86

Diagonals 0.80–0.89; off-diagonals 0.04–0.25. The bridge is strongly game-conditioned.

Within-game across-emission consistency (cosine between per-emission mean bridge tokens, off-diagonal):

Game	off-diag mean	off-diag std
MsPacman	+0.951	0.019
RoadRunner	+0.938	0.027
Q*bert	+0.918	0.025
Seaquest	+0.920	0.032
River Raid	+0.968	0.019
SpaceInvaders	+0.961	0.032
Enduro	+0.956	0.027

Within-game per-emission variation is small ($1-\bar{c} \approx 3-8\%$), suggesting most of the latent capacity is used to encode the (fixed-within-episode) game identity, with a thin per-tick residual encoding strategic state. This is the basis for the "used capacity $\ll$ nominal" observation in §6.4.

Legacy MI probe (kept for completeness). A pre-revision binned plug-in MI estimate on the trained MsPacman bridge gave $I(\text{bridge; sign of future-30-tick reward}) = 0.024$ nats vs a shuffled baseline of 0.014 nats ($+0.010$, 1.4σ). Action MI was at baseline (0.087 vs 0.085, training used random-policy trajectories). We do not draw conclusions from these numbers; the diagnostics above are more informative.

G OOD KL probe: full numbers

Per-emission probe: for each (game, Stage A variant), we walk the cached seed-0 T-trajectory, take up to 20 emissions, and at each emission tick compute $\mathrm{KL}\big(p_{\text{bare-prompt}} \,\|\, p_{\text{suffixed-prompt}}\big)$ over legal actions, the argmax-change rate $\mathbf{1}[\text{argmax } p_{\text{bare}} \neq \text{argmax } p_{\text{suff}}]$, and the probability mass on the bare argmax under each prompt. The *collapse?* column marks the three games whose bare-Stage-A bridges were degraded or zeroed at deployment (SpaceInvaders, Q*bert, River Raid; the "collapsed" group of §5).

Bare Stage A:

Game	n	KL	argmax-change	$p_{\text{bare}}(a^*_{\text{bare}})$	$p_{\text{suff}}(a^*_{\text{bare}})$	collapse?
MsPacman	20	0.54 ± 0.47	0.85	0.55	0.19	no
Seaquest	20	1.00 ± 0.28	0.70	0.29	0.10	no
RoadRunner	8	0.39 ± 0.14	0.75	0.52	0.24	no
SpaceInvaders	19	0.29 ± 0.15	0.79	0.49	0.24	yes
Q*bert	20	0.25 ± 0.22	0.90	0.41	0.20	yes
River Raid	20	0.24 ± 0.06	0.70	0.24	0.11	yes
group means: non-collapsed		0.64	0.77	0.45	0.17	
group means: collapsed		0.26	0.80	0.38	0.18	

Robust Stage A (suffix-prob=0.5):

Game	n	KL	argmax-change	$p_{\text{bare}}(a^*_{\text{bare}})$	$p_{\text{suff}}(a^*_{\text{bare}})$	collapse?
MsPacman	20	0.40 ± 0.23	0.30	0.42	0.52	no
Seaquest	20	0.34 ± 0.19	0.90	0.30	0.13	no
RoadRunner	8	0.11 ± 0.07	0.12	0.68	0.59	no
SpaceInvaders	19	0.19 ± 0.08	0.89	0.37	0.19	yes
Q*bert	20	0.11 ± 0.07	0.50	0.36	0.33	yes
River Raid	20	0.26 ± 0.13	0.65	0.24	0.20	yes
group means: non-collapsed		0.28	0.44	0.47	0.41	
group means: collapsed		0.19	0.68	0.32	0.24	

Read. (1) Under bare Stage A, the suffix flips the argmax on 70–90 % of emissions on *every* game; the OOD effect is large and uniform. (2) KL is *lower* on the collapsed games (0.26 vs 0.64), against the naive prediction. This is because the bare head on collapsed games is less peaky to start with, so the same argmax flip moves less probability mass. KL is therefore not the right summary statistic for the OOD effect—argmax-change is. (3) Robust Stage A cuts argmax-change cleanly on two of the three games where bare was already non-degenerate (RoadRunner 75→12, MsPacman 85→30; Seaquest is the exception at 70→90, matching its deployment harm under robust Stage A), confirming the fix targets the right thing where it works. On collapsed games robust Stage A reduces KL but only partially reduces argmax-change (Q*bert 90→50; River Raid 70→65), consistent with the asymmetric rescue we observed in scoring (§5).

H Decoder ablation on Q*bert (full numbers)

Backing data for §4.3. Robust Stage A, $n = 12$ episodes per cell (3 seeds $\times$ 4 episodes), ≤ 500 ticks per episode, multinomial sampling from $\text{softmax}(\text{logits}/\tau)$ over legal actions.

Decoder	Strategy	mean±std	median	min	max
greedy (paper)	F	25.0 ± 0.0	25.0	25	25
greedy (paper)	T	125.0 ± 0.0	125.0	125	125
greedy (paper)	L	50.0 ± 0.0	50.0	50	50
sample, $\tau = 0.5$	F	16.7 ± 11.8	25.0	0	25
sample, $\tau = 0.5$	T	139.6 ± 66.5	125.0	50	250
sample, $\tau = 0.5$	L	125.0 ± 66.9	125.0	50	250
sample, $\tau = 1.0$	F	66.7 ± 71.7	50.0	0	225
sample, $\tau = 1.0$	T	87.5 ± 65.0	75.0	25	250
sample, $\tau = 1.0$	L	$\mathbf{250.0 \pm 155.8}$	200.0	50	650

Significance tests, L vs T:

- $\tau = 0.5$: Welch $t = 0.51$, $p = 0.61$ (n.s.). $L \approx T$.
- $\tau = 1.0$: Welch $t \approx 3.2$, $p \approx 0.006$. L beats T by $2.9\times$.

Reading. Under greedy, every episode is deterministic given the env seed; with a small action space and consistent argmax, all 12 episodes per cell hit the same trajectory (std=0). The bridge's small per-emission variance (cosine 0.92–0.97 between emissions, Appendix F) is below the threshold needed to flip an argmax. Multinomial sampling exposes this variance: as τ increases, L's mean grows ($50 \to 125 \to 250$) while T's plateaus or declines ($125 \to 140 \to 88$). The bridge's per-tick information advantage over text is therefore real but *not visible* under greedy decoding on Q*bert.

I Longer T-suffix ablation (full numbers)

Backing data for §6.2. Same models, Stage A bare checkpoints, $n = 12$ per cell (3 seeds $\times$ 4 episodes), ≤ 500 ticks per episode. "Window" is the number of most-recent slow emissions concatenated into the T suffix (separated by a single space); the bridge L is unchanged.

Game	Window	mean±std	median	min	max
MsPacman	$w = 1$ (paper)	407.5 ± 92.2	385.0	270	600
MsPacman	$w = 2$	378.3 ± 117.5	375.0	130	680
MsPacman	$w = 3$	351.7 ± 87.7	335.0	240	560
RoadRunner	$w = 1$ (paper)	608.3 ± 250.3	650.0	0	900
RoadRunner	$w = 2$	8.3 ± 27.6	0.0	0	100
RoadRunner	$w = 3$	0.0 ± 0.0	0.0	0	0

Reading. On MsPacman, doubling/tripling the T budget gives a small monotonic decline ($-14\,\%$ from $w = 1$ to $w = 3$). On RoadRunner, $w = 2$ collapses to near-zero and $w = 3$ collapses to exact zero on every episode. Older emissions appear to be stale and to mislead the action head; the fast model does not learn an implicit time-ordering of the concatenated emissions. The L scores are unchanged by w (L uses only the latest emission), so the $L - T$ gap widens monotonically with w as T collapses.

J Sampling on other greedy-degenerate games (full numbers)

Backing data for the "greedy-decoding bias is general" claim in §4.3. Action-policy=sample at $\tau = 1.0$, $n = 12$ per cell, ≤ 500 ticks per episode. The variant is robust Stage A except for the **Seaquest (bare)** block, which is the *headline* Seaquest variant (§4) and the most consequential row: its greedy $L > T$ win does not survive sampling.

Game (variant)	Strategy	Greedy (paper)	Sample $\tau = 1.0$
Seaquest (bare)	F	41.7 ± 19.9	63.3 ± 35.0
Seaquest (bare)	T	63.3 ± 11.5	$\mathbf{143.3 \pm 37.0}$
Seaquest (bare)	L	$\mathbf{80.0 \pm 0.0}$	135.0 ± 40.1
SpaceInvaders (robust)	F	107.1 ± 62.4	106.7 ± 26.0
SpaceInvaders (robust)	T	18.3 ± 18.6	107.5 ± 19.2
SpaceInvaders (robust)	L	15.0 ± 0.0	$\mathbf{116.7 \pm 14.5}$
Seaquest (robust)	F	20.0 ± 0.0	$\mathbf{110.0 \pm 38.7}$
Seaquest (robust)	T	0.0 ± 0.0	41.7 ± 37.8
Seaquest (robust)	L	0.0 ± 0.0	33.3 ± 27.5
Enduro (robust)	F	0.8 ± 1.1	3.2 ± 2.5
Enduro (robust)	T	4.9 ± 5.9	0.2 ± 0.8
Enduro (robust)	L	5.8 ± 2.6	0.6 ± 1.4

Reading. **Seaquest (bare) is the consequential row**: under greedy $L = 80 \pm 0 > T = 63$ (a $+26\%$ headline win), but the L cell is deterministic; under sampling both bridges roughly double off F ($F: 42 \to 63$, $T: 63 \to 143$, $L: 80 \to 135$) and the ordering flips to $L \approx T$ with the Text Bridge nominally ahead (Welch $p = 0.60$, MWU $p = 0.73$). The slow channel still clearly helps ($T, L \gg F$, $p < 10^{-3}$), but the Latent-over-Text margin was a greedy artifact—greedy had under-counted T here more than L, the opposite of Q*bert. SpaceInvaders is the clearest example of greedy under-counting the bridge: under greedy both bridges are at ∼15–18 (far below F=107), suggesting bridge-induced collapse; under sampling all three strategies cluster around 107–117. The bridges were never broken on SpaceInvaders—greedy was misrepresenting them. Seaquest/robust shows the same pattern less dramatically: T=L=0 (greedy) → 33–42 (sampling); the bridges are still below F=110 but no longer collapsed. Enduro inverts the pattern: under sampling the bridges *drop* (T 5→0.2, L 6→0.6), so Enduro's greedy advantage was real but fragile. The general lesson is that greedy decoding at small action-space scale produces deterministic trajectories that can mask both the bridge's signal and the bridge's actual failure modes *and* the sign of the L-vs-T comparison; comparing F/T/L should be done under a sampling decoder when per-episode variance is required for meaningful statistics.

K Full decoder sweep (greedy / $\tau \in \{0.3, 0.5, 0.7, 1.0, 1.5\}$)

Backing data for §4.4. We re-ran the reported-variant cells at greedy and five sampling temperatures on the *same* Stage A/C checkpoints, $n = 12$ (3×4), ≤ 500 ticks. Greedy reproduces the headline exactly on MsPacman, River Raid, and Seaquest, and F reproduces on every game (it never invokes the slow model)—so the greedy-vs-sampling difference is a genuine decoder effect, not checkpoint or code drift. The per-decoder means:

Game	decoder	F	T	L	L vs T
MsPacman	greedy	256	408	628	L>T
	τ=0.3	268	343	353	L>T
	τ=0.5	358	440	360	L<T
	τ=0.7	347	342	339	L<T
	τ=1.0	348	292	368	L>T
RoadRunner	greedy	0	475	608	L>T
	τ=0.3	0	142	17	L<T
	τ=0.5	0	8	0	L<T
	τ=0.7	0	17	0	L<T
	τ=1.0	0	25	17	L<T
River Raid	greedy	1032	337	607	L>T
	τ=0.3	946	668	511	L<T
	τ=0.5	1015	717	562	L<T
	τ=0.7	890	646	641	L<T
	τ=1.0	867	649	531	L<T
Seaquest	greedy	42	63	80	L>T
	τ=0.3	50	125	130	L>T
	τ=0.5	48	128	122	L<T
	τ=0.7	58	127	137	L>T
	τ=1.0	63	143	135	L<T
	τ=1.5	63	113	97	L<T
Q*bert	greedy	25	125	50	L<T
	τ=0.3	19	110	148	L>T
	τ=0.5	19	200	162	L<T
	τ=0.7	38	138	123	L<T
	τ=1.0	108	177	188	L>T
	τ=1.5	88	242	169	L<T
Enduro	greedy	3	3	2	L<T
	τ=0.5	4	3	1	L<T
	τ=1.0	3	0	1	L>T
SpaceInvaders	greedy	107	18	15	L<T
	τ=0.5	115	104	109	L>T
	τ=1.0	135	162	142	L<T

The $L > T$ advantage is greedy-specific. On every game, the greedy $L > T$ margin vanishes under *any* fixed sampling temperature: $L \approx T$ (MsPacman, Seaquest) or significant $T > L$ (River Raid at $\tau = 1.0$; RoadRunner collapses to the $F = 0$ floor). The latent's small per-emission variance (§6.4) shifts the deterministic argmax but is swamped once the decoder samples, so the Latent Bridge yields a better *greedy policy* rather than a more robustly informative channel.

RoadRunner greedy is run-to-run-fragile in magnitude. RoadRunner greedy L is unstable across runs of the same host (a single RTX Pro 6000, Appendix E): this decoder-sweep run gives a tight $L = 608 \pm 29$—the value we report as canonical—while an earlier run scored $L = 967$. Because F reproduces exactly and only the slow-model-dependent cells drift, and only on the one game whose $F = 0$ makes the score hinge entirely on the slow emission triggering one

precise action, the cause is floating-point nondeterminism in the (greedy, `do_sample=False`) slow-model generation across runs (batched vLLM generation is not bitwise-deterministic). The $L > T$ *direction* is robust (608 vs 475); the absolute magnitude is not.

L Held-out decoder selection protocol

For the best-achievable comparison (§4.4, Table 1) we treat the action decoder as a deployment hyperparameter and select it per (game, channel) the honest way—tune on held-in data, report on held-out—using leave-one-seed-out over the 3 ALE seeds (4 episodes each):

- For each held-out seed s: pick the decoder with the highest mean over the *other* two seeds (8 episodes), then record that decoder's 4 episodes on seed s.
- The reported best-achievable cell pools the 3 held-out folds (12 episodes, each seed held out once). L vs T uses Welch's t and Mann–Whitney on the two pooled held-out vectors.

This is an unbiased estimate of "tune the decoder, then deploy," unlike an oracle max over decoders on the same episodes (which we report alongside as an optimistic upper bound). In practice the selection is stable—held-out ≈ oracle on most cells (e.g. MsPacman-L picks greedy on all three folds)—so the unbiased estimate sits close to the optimistic one. The selection procedure and table generator live under `scripts/` in the released code.

M Bridge replacement control (full numbers)

Backing data for §6.3. MsPacman bare, Stage A bare checkpoint, $n = 12$ per cell, greedy decoding (paper default). Three L variants compared against F: "zero" replaces the 8 trained bridge tokens with 8 zero vectors, "random" replaces them with random Gaussian vectors rescaled to per-position L2 norm ≈ 64 (matching trained), "trained" is the unmodified projection output.

Variant	mean±std ($n = 12$)	median	vs F	p (Welch, vs trained)
F (no bridge)	256 ± 25	250	—	—
L_{zero}	379 ± 95	395	+48%	0.04
L_{random}	387 ± 161	365	+51%	0.05
L_{trained}	$\mathbf{628 \pm 356}$	550	+145%	—

Reading. L_{zero} *vs* L_{random}: Welch $t = 0.15$, $p = 0.88$ (n.s.)—essentially the same. So the architectural lift over F comes from *having 8 prepended slots*, not from the slots having any particular content. $L_{trained}$ *vs* L_{random}: Welch $t \approx 2.12$, $p \approx 0.05$ (vs L_{zero}: $t \approx 2.34$, $p \approx 0.04$). The trained content is contributing a real signal, on top of the architectural lift. Decomposition on MsPacman: ∼40 % of L's advantage over F is architectural; ∼60 % is learned bridge content.

All-games sweep (the control run on every Atari game, not just MsPacman). The verdict table below is summarized in the main text (§7.3). We re-ran the trained/zero/random control on all 7 games using the *bare* Stage A head under greedy decoding—the natural common setting for a content control—with the single exception of Q*bert, run on its robust head under $\tau = 1$ sampling (its canonical decoder, since greedy Q*bert is degenerate, §4.3); $n = 12$ per cell (3×4). L_{random} uses matched per-position L2 norm. Because the control is run *bare*, the $T - F$ column below is the bare-head value, so three games (Enduro, SpaceInvaders, River Raid) use their

bare cell here even though the headline reports their robust variant—the learned-vs-control verdict does not depend on that choice. This is also an independent re-run, so its MsPacman cell ($L_{\text{trained}} = 666$) and Seaquest cell (100) differ slightly from the standalone / headline runs above (628 and 80); all agree within run-to-run noise. The Q*bert cell ($L_{\text{trained}} = 123$) likewise differs from the decoder ablation's $\tau = 1$ mean (250 ± 156, Appendix H); both are single high-variance sampling runs and the gap is within one standard deviation, but the verdict here (trained $\approx$ random) does not depend on the absolute value. "Learned" = L_{trained} exceeds $\max(L_{\text{zero}}, L_{\text{random}})$ by $>10\%$.

Game	L_{train}	L_{zero}	L_{rand}	verdict	$T-F$
RoadRunner	**608**	0	8	learned (Welch $t \approx 71$)	+475
Seaquest	**100**	28	5	learned ($t = 36$)	+22
MsPacman	**666**	408	410	learned ($t = 2.3$)	+152
Enduro	7.8	1.4	4.7	trend only (tiny, $t = 1.1$)	−3
Q*bert	123	63	117	≈random (tie)	+100
SpaceInvaders	0	148	90	*harmful*	−105
River Raid	360	1013	1003	*harmful*	−683

Reading: learned content tracks the predictor. The trained latent carries genuinely learned, behavior-relevant content—$L_{\text{trained}} \gg \max(L_{\text{zero}}, L_{\text{random}})$—on **RoadRunner, Seaquest, and MsPacman**, exactly the games where slow reasoning helps ($T > F$); on RoadRunner the Latent Bridge is *almost entirely* learned (controls drop to the floor, 0 and 8). On the games where slow reasoning does not help ($T \leq F$: River Raid, SpaceInvaders), the trained latent is *harmful*—zeroing or randomizing it scores *better*—the same inert/harmful pattern as MetaDrive (§7.1). Two off-diagonal cases are benign: Enduro's scores are too small for the comparison to be meaningful ($t = 1.1$, n.s.), and Q*bert—the one decoder-fragile game (§4.3)—shows trained≈random under sampling, consistent with its thin per-emission signal. So the MsPacman "∼60 % learned / ∼40 % architectural" decomposition is *not* universal: the learned fraction ranges from ∼100 % (RoadRunner) to negative (River Raid), and its sign is predicted by whether $T > F$. This is the mechanistic complement to the $r = 0.93$ predictor: the latent helps where—and only where—it has carried real content from a slow channel that itself beats Fast-Only.

N 30B-A3B cross-scale ablation (full numbers)

Replicates the MsPacman/bare F/T/L cells with the slow model swapped from Qwen3-VL-8B-Thinking (dense) to Qwen3-VL-30B-A3B-Thinking (MoE, ∼3B active per forward, 48 layers, hidden_dim=2048). Pipeline:

- **Stage B**: 15 T-trajectories collected with the 30B slow (∼470 emissions, ∼7200 ticks). Same prompt template as 8B.
- **Stage C v2**: re-trained projection 2048 $\rightarrow$ 4096 (∼33M parameters, same 2-layer MLP topology), 1 epoch over ∼7K samples. Final mean KL = 0.015 (*vs* 8B's ∼0.005; slightly under-converged given ∼3× less data per slow-model parameter).
- **Eval**: F/T/L on MsPacman, 6 seeds × 2 episodes per cell, $n = 12$, ≤500 ticks/episode. Run twice: greedy and sample $\tau = 1.0$.

Configuration	F	T	L	Δ_{L-T}
8B-dense, greedy (paper)	256 ± 25	408 ± 92	$\mathbf{628 \pm 356}$	**+54%** (Welch $p = 0.06$)
30B-A3B, greedy, $n = 12$	273 ± 45	312 ± 14	210 ± 0	−33% (det. artifact, Q*bert-style)
30B-A3B, sample $\tau = 1.0$, $n = 12$	309 ± 81	389 ± 209	$\mathbf{458 \pm 283}$	+18 % (Welch $p = 0.50$, $d = 0.28$)

Reading. A "bigger slow model carries more, so $L-T$ should grow with capacity" hypothesis is not supported here. With Qwen3-VL-30B-A3B as slow, the $L > T$ direction is preserved under sampling (+18 %) but the magnitude *shrinks* relative to 8B-dense (+54 %), and significance is lost ($p = 0.50$, $n = 12$). The result is consistent with several non-exclusive readings:

- **Active parameters, not total.** 30B-A3B activates only ∼3B of its 30B parameters per forward pass. A dense-30B comparison (which we cannot run on this GPU) would be the right capacity test.
- **Hidden-dim shift.** 30B-A3B has hidden_dim=2048 vs 8B's 4096, so the trainable projection maps $2048 \to 4096$ rather than $4096 \to 4096$. The same parameter budget covers a harder map.
- **Bridge under-training.** We trained on ∼470 30B emissions; the 8B paper baseline used ∼5K. Final KL 0.015 vs 0.005 is consistent with partial convergence.
- **One-game evidence.** A single game on a single cross-scale data point is not strong enough to falsify the scaling claim; we report this as a single counter-data point, not as a refutation.

The greedy result ($L = 210 \pm 0$) reproduces the deterministic-trajectory pattern we documented for Q*bert under greedy in the main text—another case where greedy decoding interacts pathologically with the bridge's low per-emission variance.

O Per-game collapse triage

Across the 14 (game, variant) cells in Appendix A, five have a strategy cell at zero (the first five rows below). Robust Stage A additionally destroys one already-working cell without driving it to exactly zero (MsPacman/robust, $L \to 60$), and a separate game, Pong (outside the 14), sits at the random-policy floor (−21). Stage A val-accuracy is the load-bearing predictor of collapse, not bridge-training failure: σ of T, L at deployment correlates with bare Stage A val-acc, not with Stage C KL.

Game/variant	Collapsed cell	Diagnosis
RoadRunner/bare	$F = 0$	Stage A predicts NOOP; bridges rescue.
SpaceInvaders/bare	$T = L = 0$	Stage A OOD-brittle; robust partially rescues.
Q*bert/bare	$T = L = 0$	Stage A OOD-brittle; robust rescues to $T = 125$, $L = 50$.
Seaquest/robust	$T = L = 0$	Robust drops below bare F; use bare.
Enduro/bare	$T = 0$	Text collapses; robust gives a non-degenerate near-floor tie.
MsPacman/robust	$L \to 60$	(non-zero) Robust destroys MsPacman policy; use bare.
Pong/all	all −21	(outside 14) Stage A val-acc only 1.5× random; cannot be rescued.

P MetaDrive (non-Atari) full numbers

Setup. MetaDrive [13] at ∼10 Hz control. The fast model sees a top-down rendered 84×84 image (rendered via the pygame top-down channel, no GL—this is what makes headless end-to-end runs feasible on one GPU); 9 discrete actions (3 steer × 3 throttle). The slow model sees the structured state (speed, heading, lane offset, upcoming-turn cue derived from `navi_arrow_dir`, nearby vehicles). Stage A clones MetaDrive's built-in PPO [20] expert; Stage B collects slow emissions under expert driving; Stage C distills the latent against the (robust, suffix-aware) text teacher. Random policy scores ∼8 reward; the discrete expert scores ∼174–211.

Two task regimes. *Reactive* = default 3-block map (lane-keeping dominates). *Planning* = map `SXSXSX` (straights + X-intersections) with off-route termination, so survival requires correct turn decisions: the route expert scores 211 ± 65 vs 93 ± 23 for a constant straight+throttle policy ($n = 40$ episodes each), a +118-reward gap. Eval on the robust head: $n = 8$ (4 seeds × 2 episodes) for the planning rows, $n = 6$ (3 seeds × 2) for the reactive row.

Regime	Decoder	F	T	L
Reactive	greedy	71.2	69.5	69.5
Planning	greedy	87.8	85.1	85.1
Planning	sample $\tau = 1$	123.2	43.5	36.7

Bridge-replacement control (planning, greedy, $n = 8$). $L = 85.1$, $L_{\text{zero}} = 89.5$, $L_{\text{random}} = 97.1$: replacing the trained latent with zeros or matched-norm random vectors does *not* reduce the score (if anything it rises), i.e. the Latent Bridge is inert—it carries no behavior-relevant information. This is the opposite of MsPacman ($L_{\text{trained}} = 628 \gg L_{\text{random}} = 387$, §M), and is the diagnostic basis for calling MetaDrive a controlled negative.

Why L cannot exceed F here. Stage C trains L to match the text teacher's action distribution (objective $\mathrm{KL}(\pi_L \| \pi_T)$), so L's ceiling is T. On MetaDrive $T \leq F$ in every configuration we ran—both task regimes (reactive lane-keeping, planning intersections), both decoders (greedy, sampling), and both Stage A heads: the bare head, where the text channel collapses ($T = 17.2$ vs $F = 73.7$), and the robust head, where T sits just below F in the tabulated reactive (69.5 vs 71.2), planning-greedy (85.1 vs 87.8), and planning-sampling (43.5 vs 123.2) cells—so the best attainable latent is $L \approx F$. Fixing the teacher (robust, suffix-aware head, expert-driven data, KL converged to 0.004) makes the distillation *clean*—$L = T$ to the decimal under greedy—but cannot raise a ceiling set by a teacher that does not beat Fast-Only. The slow model's frame-grounded, ∼1.5 s reasoning simply does not produce better driving decisions than the reactive policy, even when the task demands route planning.

Robustness of the predictor. The $(T-F, L-F)$ correlation (Figure 8) is robust to both variant selection and individual games. *Variant selection:* $r = 0.93$ over the 8 reported-variant cells and $r = 0.96$ over all 16 (game, variant) cells with no selection—the 16 are the 14 bare/robust Atari cells, a SpaceInvaders expert-data cell (§5), and MetaDrive—so the choice of which Stage A variant to headline does not create the relationship. *Individual games* (on the 8 reported-variant cells): leave-one-out Pearson r stays in $[0.89, 0.95]$ for all 8 deletions (dropping the RoadRunner

high-leverage point gives $r = 0.89$); the rank correlation is Spearman $\rho = 0.98$; a game-level bootstrap (10,000 resamples) gives a 95 % CI of $[0.81, 1.0]$; and excluding both dynamic-range extremes (RoadRunner and River Raid) still gives $r = 0.84$ ($n = 6$). *Decoder:* the predictor is about $L-F$ vs $T-F$, not L vs T, so the Seaquest decoder-fragility finding (§4.3) does not touch it—Seaquest stays in the upper-right helps-quadrant under both decoders ($T-F, L-F = (+22, +38)$ greedy, $(+80, +72)$ sampling), if anything reinforcing the relationship. What the decoder moves is the *secondary* latent-vs-text tally, not the task-level predictor.